Solution: Problem Challenge 2: Scheduling Tasks

Problem Statement

You are given a list of tasks that need to be run, in any order, on a server. Each task will take one CPU interval to execute but once a task has finished, it has a cooling period during which it can't be run again. If the cooling period for all tasks is 'K' intervals, find the minimum number of CPU intervals that the server needs to finish all tasks.

If at any time the server can't execute any task then it must stay idle.

Example 1:

```
Input: [a, a, a, b, c, c], K=2
Output: 7
Explanation: a -> c -> b -> a -> c -> idle -> a
```

Example 2:

```
Input: [a, b, a], K=3
Output: 5
Explanation: a -> b -> idle -> idle -> a
```

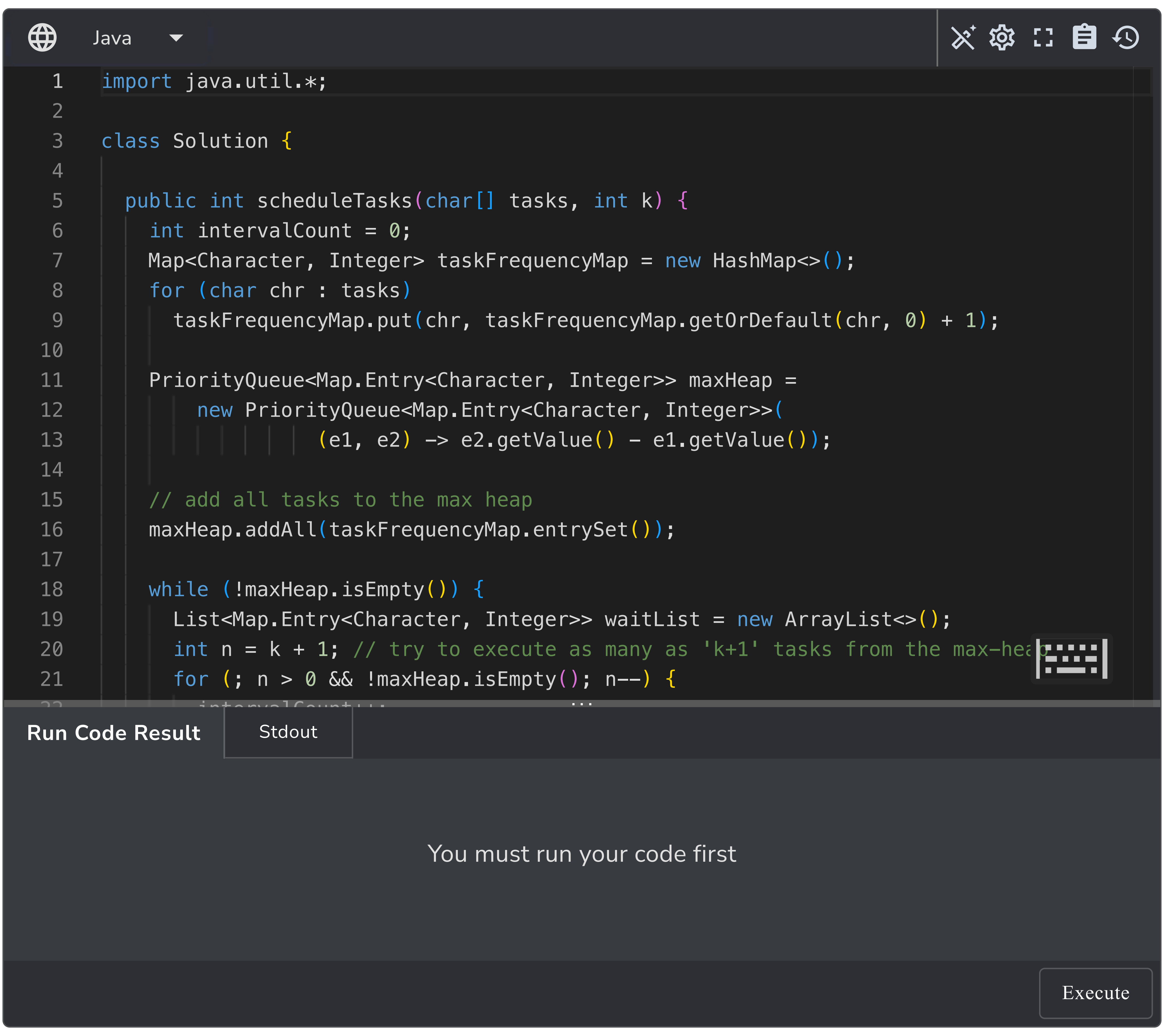
Solution

This problem follows the `Top 'K' Numbers` pattern and is quite similar to `Rearrange String K Distance Apart`. We need to rearrange tasks such that same tasks are 'K' distance apart.

Following a similar approach, we will use a **Max Heap** to execute the highest frequency task first. After executing a task we decrease its frequency and put it in a waiting list. In each iteration, we will try to execute as many as `k+1` tasks. For the next iteration, we will put all the waiting tasks back in the **Max Heap**. If, for any iteration, we are not able to execute `k+1` tasks, the CPU has to remain idle for the remaining time in the next iteration.

Code

Here is what our algorithm will look like:



```java
import java.util.*;

class Solution {

  public int scheduleTasks(char[] tasks, int k) {
    int intervalCount = 0;
    Map<Character, Integer> taskFrequencyMap = new HashMap<>();
    for (char chr : tasks)
      taskFrequencyMap.put(chr, taskFrequencyMap.getOrDefault(chr, 0) + 1);

    PriorityQueue<Map.Entry<Character, Integer>> maxHeap =
        new PriorityQueue<Map.Entry<Character, Integer>>(
            (e1, e2) -> e2.getValue() - e1.getValue());

    // add all tasks to the max heap
    maxHeap.addAll(taskFrequencyMap.entrySet());

    while (!maxHeap.isEmpty()) {
      List<Map.Entry<Character, Integer>> waitList = new ArrayList<>();
      int n = k + 1; // try to execute as many as 'k+1' tasks from the max-hea
      for (; n > 0 && !maxHeap.isEmpty(); n--) {
```

You must run your code first

Time Complexity

The time complexity of the above algorithm is O(N∗logN) where 'N' is the number of tasks. Our `while loop` will iterate once for each occurrence of the task in the input (i.e. 'N') and in each iteration we will remove a task from the heap which will take O(logN) time. Hence the overall time complexity of our algorithm is O(N∗logN).

Space Complexity

The space complexity will be O(N), as in the worst case, we need to store all the 'N' tasks in the **HashMap**.

← Previous
Problem Challenge 2: Scheduling Tasks (Hard)

Next →
Problem Challenge 3: Frequency Stack (Hard)

☐ Mark as Completed